

Roteiro do Vídeo de Demonstração

Projeto: All Invest · **Lab 4:** Investment Component e Portfolio Component · **Duração prevista:** cerca de 7min30s (limite: 5 a 8 min)

Integrantes:

- | | |
|---|---|
| 1. Danilo Henrique de Paulo 22.222.008-9 | 4. Pedro Henrique da Fonseca do Nascimento 22.123.099-8 |
| 2. Jônatas da Silva Gonçalves 22.123.095-6 | 5. Vinícius do Nascimento Generoso 22.123.103-8 |
| 3. Kaynã de Deus Ferreira da Silva 22.123.096-4 | 6. Wallace dos Santos Izidoro 22.123.104-6 |

Cada seção numerada (1 a 6) cobre o item correspondente do enunciado. As falas são sugestões para ler com naturalidade; ajustem as palavras ao estilo de cada apresentador. Com seis integrantes, cada um pode apresentar uma das seções numeradas.

Antes de gravar

1. Executar `database/migrations/002_create_portfolios.sql` no SQL Editor do Supabase (uma única vez). Sem isso, as rotas `/api/portfolios` retornam erro no banco real.
2. Subir o backend: `cd backend, .\venv\Scripts\Activate.ps1, uvicorn src.main:app --reload --port 8000`.
3. Abrir `http://localhost:8000/docs`, clicar em **Authorize** e preencher **ApiKeyAuth** com a `API_KEY`. Fazer `POST /api/v1/auth/login` com uma conta de teste e colar o `access_token` em **BearerAuth**.
4. Usar uma conta sem VALE3 na carteira (ou remover antes com `DELETE /api/stocks/VALE3`) e sem carteiras chamadas "Dividendos" ou "Longo Prazo".
5. Deixar abertos no VS Code: `interfaces/components.py`, `services/portfolio_service.py`, `services/investment_service.py`, `dependencies/components.py`, `routers/portfolios.py` e a migração. Aumentar a fonte do editor e do terminal.
6. Deixar o terminal em `backend/` com o `venv` ativo e `pip install -r requirements-dev.txt` já executado.
7. Ter à mão o diagrama do Lab 3 e o `Especificacao_Componentes.pdf` (Figura 1).

Cenas

Tempo	Seção	O que mostrar na tela	Fala sugerida
0:00 – 0:30	Abertura	Capa do <code>Especificacao_Componentes.pdf</code> ou <code>README</code> do repositório no GitHub.	Olá! Somos o grupo do All Invest, uma aplicação para centralizar investimentos em ações da B3. Neste vídeo mostramos a implementação de dois componentes da arquitetura que modelamos no Lab 3: o Investment Component e o Portfolio Component.

Tempo	Seção	O que mostrar na tela	Fala sugerida
0:30 – 1:30	1. Arquitetura implementada	Diagrama de componentes do Lab 3; em seguida, a Figura 1 do Especificacao_Componentes.pdf (recorte com os dois componentes).	No Lab 3 identificamos sete componentes. Conforme combinado em aula, implementamos o Investment, que registra compras e vendas e consolida as posições, e o Portfolio, que gerencia as carteiras do usuário. O Investment requer a interface IPortfolioService, que é fornecida pelo Portfolio. Ele também usa o Market Data, que já existia, para validar o ticker na B3. Essa dependência não estava no diagrama do Lab 3, e a justificativa está na documentação.
1:30 – 2:45	2. Interfaces e operações	VS Code: backend/src/interfaces/components.py. Rolar por IPortfolioService e IInvestmentService, mostrando as docstrings.	As interfaces são Protocols do Python. A IPortfolioService fornece criar, listar, consultar, editar e remover carteira e associar um investimento a uma carteira; são as operações do Lab 3, com nomes em inglês. A IInvestmentService fornece registrar operação, consultar o histórico e as posições, associar uma posição a uma carteira e remover um investimento. Em cada operação, a docstring traz o contrato: as pré-condições, como a carteira pertencer ao usuário, e as pós-condições, como o ativo ficar em exatamente uma carteira.
2:45 – 3:45	3. Código	services/portfolio_service.py (create_portfolio, associate_investment); services/investment_service.py (construtor e create_trade); routers/portfolios.py; database/migrations/002_create_portfolios.sql.	O Portfolio Component está no PortfolioService. Ele valida nome duplicado e devolve 404 para carteira de outro usuário. O Investment Component está no InvestmentService; notem que o construtor recebe um IPortfolioService, não a classe concreta. As rotas REST ficam em routers/portfolios.py e routers/stocks.py. Para os dados do Portfolio criamos as tabelas portfolios e portfolios_stocks, em uma migração que não altera nenhuma tabela existente.
3:45 – 5:00	4. Integração	dependencies/components.py (função get_investment_service). Depois, Swagger em http://localhost:8000/docs: passos A a D do Apêndice.	A ligação entre os componentes é feita aqui, pela injeção de dependências do FastAPI: o PortfolioService é criado e entregue ao InvestmentService como IPortfolioService. Vamos ver isso funcionando. Primeiro criamos a carteira Dividendos. Depois registramos uma compra de VALE3 informando essa carteira: o Investment chama get_portfolio para validar a carteira, valida o ticker no Market Data, grava a operação e chama associate_investment. Ao consultar a carteira filtrando por ela, a posição aparece com o nome da carteira. Se informarmos uma carteira que não existe, o contrato é respeitado: 404, e nada é gravado.

Tempo	Seção	O que mostrar na tela	Fala sugerida
5:00 – 6:15	5. Testes	Terminal em backend/ com o venv ativo. Executar os comandos 1 a 4 da seção "Comandos dos testes".	Temos 45 testes automatizados, que usam SQLite em memória e um stub do Market Data, sem depender do Supabase nem da internet. Este é um teste do Portfolio: nome repetido retorna 409. Este é do Investment, isolado com um stub da interface do Portfolio: ele confere a quantidade e o preço médio depois de compras e uma venda. Este é de integração, com os dois componentes reais: registrar uma operação com carteira deixa o ativo associado no Portfolio. Por fim, a suíte completa: 45 aprovados. A saída fica salva em docs/evidencias.
6:15 – 7:15	6. Resultado	Swagger: passos E a H do Apêndice. Opcional: painel do frontend em <code>http://localhost:5173</code> mostrando que a carteira continua funcionando.	Para fechar, criamos a carteira Longo Prazo e movemos a VALE3 para ela; a posição já volta com a nova carteira. Editamos o nome da carteira e, ao removê-la, as operações continuam registradas: a posição aparece sem carteira, como definido na pós-condição de <code>removeCarteira</code> . O frontend existente continua funcionando, porque os campos novos são opcionais.
7:15 – 7:30	Encerramento	README no GitHub (seções de interfaces, comunicação e testes).	Todo o código, os testes, as evidências e a especificação estão no repositório, e o README explica como executar. Obrigado!

Comandos dos testes (seção 5)

```
cd backend
.\.venv\Scripts\Activate.ps1

# 1) Teste do Portfolio Component
python -m pytest tests/test_portfolio_component.py -k nome_repetido -v

# 2) Teste do Investment Component (Portfolio substituído por stub)
python -m pytest tests/test_investment_component.py -k preco_medio -v

# 3) Teste de integração Investment + Portfolio
python -m pytest tests/test_integration_investment_portfolio.py -k associa_ativo_no_portfolio -v

# 4) Suíte completa (esperado: 45 passed)
python -m pytest -v
```

Cada filtro `-k` seleciona exatamente um teste (1 passed). Em cada execução, destaque o **PASSED** e, na suíte completa, a linha final **45 passed**.

Apêndice – Passos no Swagger (seções 4 e 6)

Se os ids gerados forem outros, use os retornados na resposta do passo A e do passo E.

Passo	Endpoint	Corpo da requisição (JSON)	Resultado esperado
A	POST /api/portfolios	{"portfolio_name": "Dividendos", "description": "Ações pagadoras de dividendos"}	201; anote o portfolio_id (ex.: 1).
B	POST /api/stocks	{"stock_name": "VALE3", "stock_quantity": 100, "stock_price": 60.50, "broker_id": 1, "trade_side": "BUY", "portfolio_id": 1}	201 com a operação gravada.
C	GET /api/stocks/carteira?portfolio_id=1	—	Posição de VALE3 com portfolio_name: "Dividendos".
D	POST /api/stocks	{"stock_name": "VALE3", "stock_quantity": 1, "stock_price": 60.50, "broker_id": 1, "trade_side": "BUY", "portfolio_id": 999999}	404 "Carteira não encontrada."
E	POST /api/portfolios	{"portfolio_name": "Longo Prazo"}	201; anote o id (ex.: 2).
F	PUT /api/stocks/VALE3/portfolio	{"portfolio_id": 2}	200; a posição volta com portfolio_name: "Longo Prazo".
G	PATCH /api/portfolios/2	{"portfolio_name": "Aposentadoria"}	200 com o nome alterado.

Passo	Endpoint	Corpo da requisição (JSON)	Resultado esperado
H	DELETE /api/portfolios/2 e depois GET /api/stocks/carteira	—	200; VALE3 continua na lista, com portfolio_id: null.

Checklist do enunciado

Item exigido	Onde aparece
1. Arquitetura implementada (componentes selecionados)	Seção 1 (0:30 – 1:30)
2. Interfaces e operações (fornecidas e requeridas)	Seção 2 (1:30 – 2:45)
3. Código (componentes, interfaces e operações)	Seção 3 (2:45 – 3:45)
4. Integração (um componente usando o outro)	Seção 4 (3:45 – 5:00): passos A a D
5. Testes (um de cada componente e um de integração)	Seção 5 (5:00 – 6:15): comandos 1, 2 e 3, e a suíte completa
6. Resultado (funcionamento da solução)	Seção 6 (6:15 – 7:15): passos E a H